Specification of Source §1 Typed—2023 edition

Martin Henz, Jingjing Zhao

National University of Singapore
School of Computing

May 1, 2026

The language Source is the official language of the textbook *Structure and Interpretation of Computer Programs*, JavaScript Adaptation. Source is a sublanguage of ECMAScript 2018 (9th Edition) and defined in the documents titled "Source §x", where x refers to the respective textbook chapter.
Source §1 Typed is a variant of Source §1 that introduces type syntax and type checking.

1 Syntax

A Source program is a *program*, defined using Backus-Naur Form[1] as follows:

program	::=	*import-directive*... *type-alias*... *statement*...	program
import-directive	::=	**import** { *import-names* } **from** *string* ;	import directive
import-names	::=	ϵ | *import-name* (, *import-name*)...	import name list
import-name	::=	*name* | *name* **as** *name*	import name
type-alias	::=	**type** *name*[< *name* (, *name*)... >] = *alias-type*;	type alias declaration
statement	::=	**const** *name*[: *type*] = *expression* ;	constant declaration
	|	**function** *name* (*names*)[: *type*] *block*	function declaration
	|	**return** *expression* ;	return statement
	|	*if-statement*	conditional statement
	|	*block*	block statement
	|	*expression* ;	expression statement
	|	**debugger** ;	breakpoint
names	::=	ϵ | *name*[: *type*] (, *name*[: *type*])...	name list
typed-names	::=	ϵ | *name*: *type* (, *name*: *type*)...	name list (typed)
if-statement	::=	**if** (*expression*) *block*	
		else (*block* | *if-statement*)	conditional statement
block	::=	{ *statement*... }	block statement
expression	::=	*number*	primitive number expression
	|	**true** | **false**	primitive boolean expression
	|	*string*	primitive string expression
	|	*name*	name expression
	|	*expression binary-operator expression*	binary operator combination
	|	*unary-operator expression*	unary operator combination

[1] We adopt Henry Ledgard's BNF variant that he described in *A human engineered variant of BNF*, ACM SIGPLAN Notices, Volume 15 Issue 10, October 1980, Pages 57-62. In our grammars, we use **bold** font for keywords, *italics* for syntactic variables, ϵ for nothing, $x \mid y$ for x or y, [x] for an optional x, x... for zero or more repetitions of x, and (x) for clarifying the structure of BNF expressions.

	|	*expression binary-logical expression*	logical composition
	|	*expression* **(** *expressions* **)**	function application
	|	(*names*) **=>** *expression*	lambda expression (expr. body)
	|	(*names*) **=>** *block*	lambda expression (block body)
	|	*expression* **?** *expression* **:** *expression*	conditional expression
	|	**(** *expression* **)**	parenthesised expression
	|	*expression* **as** *type*	as expression
binary-operator	::=	**+** | **-** | ***** | **/** | **%** | **===** | **!==**	
	|	**>** | **<** | **>=** | **<=**	binary operator
unary-operator	::=	**!** | **-**	unary operator
binary-logical	::=	**&&** | **||**	logical composition symbol
expressions	::=	ϵ | *expression* (**,** *expression*)...	argument expressions
type	::=	`number` | `boolean` | `string`	
		| `undefined` | `void` | `any`	basic type
	|	*number* | *string* | **true** | **false**	literal type
	|	*name*[< *type* (**,** *type*)... >]	type reference
	|	(*typed-names*) **=>** *type*	function type
	|	*type* **|** *type*	union type
alias-type	::=	`number` | `boolean` | `string`	
		| `undefined` | `void` | `any`	basic type
	|	*number* | *string* | **true** | **false**	literal type
	|	*name*[< *alias-type* (**,** *alias-type*)... >]	type reference
	|	(*typed-names*) **=>** *alias-type*	function type
	|	*alias-type* **|** *alias-type*	union type
	|	*name*	type parameter

Restrictions

- Return statements are only allowed in bodies of functions.
- There cannot be any newline character between **`return`** and *expression* in return statements.[2]
- There cannot be any newline character between (*name* | **`(`** *parameters* **`)`**) and **`=>`** in function definition expressions.[3]
- Implementations of Source are allowed to treat function declaration as syntactic sugar for constant declaration.[4] Source programmers need to make sure that functions are not called before their corresponding function declaration is evaluated.

Import directives

Import directives allow programs to import values from modules and bind them to names, whose scope is the entire program in which the import directive occurs. Import directives can only appear at the top-level. All names that appear in import directives must be distinct, and must also be distinct from all top-level variables. The Source specifications do not specify how modules are programmed.

Logical Composition

Conjunction

$$\textit{expression}_1 \ \texttt{\&\&} \ \textit{expression}_2$$

stands for

$$\textit{expression}_1 \ \texttt{?} \ \textit{expression}_2 \ \texttt{:} \ \texttt{false}$$

Disjunction

$$\textit{expression}_1 \ \texttt{||} \ \textit{expression}_2$$

stands for

$$\textit{expression}_1 \ \texttt{?} \ \texttt{true} \ \texttt{:} \ \textit{expression}_2$$

Names

Names[5] start with _, `$` or a letter[6] and contain only _, `$`, letters or digits[7]. Restricted words[8] are not allowed as names.
Valid names are `x`, `_45`, `$$` and π, but always keep in mind that programming is communicating and that the familiarity of the audience with the characters used in names is an important aspect of program readability.

Numbers

We use decimal notation for numbers, with an optional decimal dot. "Scientific notation" (multiplying the number with 10^x) is indicated with the letter `e`, followed by the exponent x. Examples for numbers are `5432`, `-5432.109`, and `-43.21e-45`.

[2] Source inherits this syntactic quirk of JavaScript.

[3] ditto

[4] ECMAScript prescribes "hoisting" of function declarations to the beginning of the surrounding block. Programs that rely on this feature will run fine in JavaScript but might encounter a runtime error "Cannot access name before initialization" in a Source implementation.

[5] In ECMAScript 2020 (9th Edition), these names are called *identifiers*.

[6] By *letter* we mean Unicode letters (L) or letter numbers (Nl).

[7] By *digit* we mean characters in the Unicode categories Nd (including the decimal digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9), Mn, Mc and Pc.

[8] By *restricted word* we mean any of: **`arguments`**, **`await`**, **`break`**, **`case`**, **`catch`**, **`class`**, **`const`**, **`continue`**, **`debugger`**, **`default`**, **`delete`**, **`do`**, **`else`**, **`enum`**, **`eval`**, **`export`**, **`extends`**, **`false`**, **`finally`**, **`for`**, **`function`**, **`if`**, **`implements`**, **`import`**, **`in`**, **`instanceof`**, **`interface`**, **`let`**, **`new`**, **`null`**, **`package`**, **`private`**, **`protected`**, **`public`**, **`return`**, **`static`**, **`super`**, **`switch`**, **`this`**, **`throw`**, **`true`**, **`try`**, **`typeof`**, **`var`**, **`void`**, **`while`**, **`with`**, **`yield`**. These are all words that cannot be used without restrictions as names in the strict mode of ECMAScript 2020.

Strings

Strings are of the form `"`*double-quote-characters*`"`, where *double-quote-characters* is a possibly empty sequence of characters without the character `"` and without the newline character, of the form `'`*single-quote-characters*`'`, where *single-quote-characters* is a possibly empty sequence of characters without the character `'` and without the newline character, and of the form `` `*backquote-characters*` ``, where *backquote-characters* is a possibly empty sequence of characters without the character `` ` ``. Note that newline characters are allowed as *backquote-characters*.
The following characters can be represented in strings as given:

- horizontal tab: `\t`
- vertical tab: `\v`
- nul char: `\0`
- backspace: `\b`
- form feed: `\f`
- newline: `\n`
- carriage return: `\r`
- single quote: `\'`
- double quote: `\"`
- backslash: `\\`

Unicode characters can be used in strings using `\u` followed by the hexadecimal representation of the unicode character, for example `'\uD83D\uDC04'`.

Comments

In Source, any sequence of characters between "`/*`" and the next "`*/`" is ignored.
After "`//`" any characters until the next newline character is ignored.

2 Type System

In Source §1 Typed, the Source §1 syntax is expanded to include type syntax such as type annotations and type aliases. This allows names to be explicitly typed, and for type checks to be performed.
Support for `typeof` operations is also added to Source §1 Typed.

2.1 Type Environment

In order to keep track of the type of names in a program, we define a *type environment*, denoted by Γ. More formally, the partial function Γ from names to types expresses a context, in which a name x is associated with type $\Gamma(x)$.
We define a relation $\Gamma[x \leftarrow t]\Gamma'$ on type environments Γ, names x, types t, and type environments Γ', which constructs a type environment that behaves like the given one, except that the type of x is t. More formally, if $\Gamma[x \leftarrow t]\Gamma'$, then $\Gamma'(y)$ is t, if $y = x$ and $\Gamma(y)$ otherwise. Obviously, this uniquely identifies Γ' for a given Γ, x, and t, and thus the type environment extension relation is functional in its first three arguments.
The set of names, on which a type environment Γ is defined, is called the domain of Γ, denoted by $dom(\Gamma)$.
For each non-overloaded primitive operator, we add a binding to our initial type environment Γ_0 as follows:

$$
\begin{aligned}
\emptyset[-_2 &\leftarrow (\texttt{number}, \texttt{number}) \rightarrow \texttt{number}] \\
[* &\leftarrow (\texttt{number}, \texttt{number}) \rightarrow \texttt{number}] \\
[/ &\leftarrow (\texttt{number}, \texttt{number}) \rightarrow \texttt{number}] \\
[\% &\leftarrow (\texttt{number}, \texttt{number}) \rightarrow \texttt{number}] \\
[\&\& &\leftarrow (\texttt{boolean}, \texttt{T}) \rightarrow \texttt{boolean} \mid \texttt{T}] \\
[|| &\leftarrow (\texttt{boolean}, \texttt{T}) \rightarrow \texttt{boolean} \mid \texttt{T}] \\
[! &\leftarrow \texttt{boolean} \rightarrow \texttt{boolean}] \\
[-_1 &\leftarrow \texttt{number} \rightarrow \texttt{number}] \\
[\texttt{typeof} &\leftarrow \texttt{any} \rightarrow \texttt{string}]\Gamma_{-2}
\end{aligned}
$$

The overloaded binary primitives (with the exception of +, the handling of which will be elaborated in Typing Relations) are handled as follows:

$$
\begin{aligned}
\Gamma_{-2}[=== &\leftarrow (\texttt{string},\ \texttt{string}) \rightarrow \texttt{boolean} \mid (\texttt{number},\ \texttt{number}) \rightarrow \texttt{boolean}] \\
[!== &\leftarrow (\texttt{string},\ \texttt{string}) \rightarrow \texttt{boolean} \mid (\texttt{number},\ \texttt{number}) \rightarrow \texttt{boolean}] \\
[> &\leftarrow (\texttt{string},\ \texttt{string}) \rightarrow \texttt{boolean} \mid (\texttt{number},\ \texttt{number}) \rightarrow \texttt{boolean}] \\
[>= &\leftarrow (\texttt{string},\ \texttt{string}) \rightarrow \texttt{boolean} \mid (\texttt{number},\ \texttt{number}) \rightarrow \texttt{boolean}] \\
[< &\leftarrow (\texttt{string},\ \texttt{string}) \rightarrow \texttt{boolean} \mid (\texttt{number},\ \texttt{number}) \rightarrow \texttt{boolean}] \\
[<= &\leftarrow (\texttt{string},\ \texttt{string}) \rightarrow \texttt{boolean} \mid (\texttt{number},\ \texttt{number}) \rightarrow \texttt{boolean}]\Gamma_{-1}
\end{aligned}
$$

The Source §1 standard library functions and constants have their types defined as follows:

$$
\begin{array}{llllll}
\Gamma_{-1} & [\ \texttt{display} & \leftarrow & \texttt{any} & &] \\
 & [\ \texttt{error} & \leftarrow & \texttt{any} & &] \\
 & [\ \texttt{Infinity} & \leftarrow & \texttt{number} & &] \\
 & [\ \texttt{is_boolean} & \leftarrow & \texttt{any} & \rightarrow\ \texttt{boolean} &] \\
 & [\ \texttt{is_function} & \leftarrow & \texttt{any} & \rightarrow\ \texttt{boolean} &] \\
 & [\ \texttt{is_number} & \leftarrow & \texttt{any} & \rightarrow\ \texttt{boolean} &] \\
 & [\ \texttt{is_string} & \leftarrow & \texttt{any} & \rightarrow\ \texttt{boolean} &] \\
 & [\ \texttt{is_undefined} & \leftarrow & \texttt{any} & \rightarrow\ \texttt{boolean} &] \\
 & [\ \texttt{math_abs} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_acos} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_acosh} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_asin} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_asinh} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_atan} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_atan2} & \leftarrow & (\texttt{number}, \texttt{number}) & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_atanh} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_cbrt} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_ceil} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_clz32} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_cos} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_cosh} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_exp} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_expm1} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_floor} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_fround} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_hypot} & \leftarrow & \texttt{any} & &] \\
 & [\ \texttt{math_imul} & \leftarrow & (\texttt{number}, \texttt{number}) & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_LN2} & \leftarrow & \texttt{number} & &] \\
 & [\ \texttt{math_LN10} & \leftarrow & \texttt{number} & &] \\
 & [\ \texttt{math_log} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_log1p} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &] \\
 & [\ \texttt{math_log2} & \leftarrow & \texttt{number} & \rightarrow\ \texttt{number} &]
\end{array}
$$

$$
\begin{array}{llllll}
[& \texttt{math_LOG2E} & \leftarrow & \texttt{number} & &] \\
[& \texttt{math_log10} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{math_LOG10E} & \leftarrow & \texttt{number} & &] \\
[& \texttt{math_max} & \leftarrow & \texttt{any} & &] \\
[& \texttt{math_min} & \leftarrow & \texttt{any} & &] \\
[& \texttt{math_PI} & \leftarrow & \texttt{number} & &] \\
[& \texttt{math_pow} & \leftarrow & (\texttt{number}, \texttt{number}) & \rightarrow \texttt{number} &] \\
[& \texttt{math_random} & \leftarrow & () & \rightarrow \texttt{number} &] \\
[& \texttt{math_round} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{math_sign} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{math_sin} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{math_sinh} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{math_sqrt} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{math_SQRT1_2} & \leftarrow & \texttt{number} & &] \\
[& \texttt{math_SQRT2} & \leftarrow & \texttt{number} & &] \\
[& \texttt{math_tan} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{math_tanh} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{math_trunc} & \leftarrow & \texttt{number} & \rightarrow \texttt{number} &] \\
[& \texttt{NaN} & \leftarrow & \texttt{number} & &] \\
[& \texttt{parse_int} & \leftarrow & (\texttt{string}, \texttt{number}) & \rightarrow \texttt{number} &] \\
[& \texttt{prompt} & \leftarrow & \texttt{string} & \rightarrow \texttt{string} &] \\
[& \texttt{get_time} & \leftarrow & () & \rightarrow \texttt{number} &] \\
[& \texttt{stringify} & \leftarrow & \texttt{any} & \rightarrow \texttt{string} &] \\
[& \texttt{undefined} & \leftarrow & \texttt{undefined} & &] \quad \Gamma_0
\end{array}
$$

In order to support the definition of type aliases, we define a separate *type alias environment*, denoted by Γ_{alias}. Unlike Γ, Γ_{alias} binds names to special *type functions* of the form $< T_1, \ldots, T_n > \rightarrow t$ where $T_1 \ldots T_n$ are type parameters t is the return type expressed in terms of $T_1 \ldots T_n$. $<>$ is used to differentiate type functions from function types, which are of the form $(t_1, \ldots, t_n) \rightarrow t$.
Since Γ and Γ_{alias} are separate environments, the same name x can be used for both variables and type aliases.

2.2 Success Types

In order for type checks to be performed in Source §1 Typed, we introduce the notion of success types.
We first define the special `any` type:

Definition 2.1 *`any` is the union of all possible types.*

Success typing in Source Typed is defined as follows:

Definition 2.2 *Type t' is a success type of type t if $\exists x(x \in t \wedge x \in t')$. Alternatively: $t \wedge t' \neq \emptyset$.*

In Source Typed, type checks are performed by checking that the actual type is a success type of the expected type. This means that type errors will be thrown if and only if a definite clash in types at runtime is detected. Given that `any` is the union of all possible types, this also means that the `any` type is guaranteed not to produce any type errors.

2.3 Typing Relations

To perform type checking on the program, typing relations are applied to every statement and expression in the program.
Names that do not have a type declared will be assumed to have the `any` type.

2.3.1 Typing Relations on Expressions

The derived type of primitive expressions is their literal type, which is an element of its corresponding basic type.

$$\frac{}{\Gamma, \Gamma_{alias} \vdash n : \textit{literal type } n} \qquad \frac{}{\Gamma, \Gamma_{alias} \vdash s : \textit{literal type } s}$$

where n denotes any literal number and s denotes any literal string.

$$\frac{}{\Gamma, \Gamma_{alias} \vdash \texttt{\textbf{true}} : \textit{literal type } \texttt{\textbf{true}}} \qquad \frac{}{\Gamma, \Gamma_{alias} \vdash \texttt{\textbf{false}} : \textit{literal type } \texttt{\textbf{false}}}$$

For names, the type must be derived from the type environment.

$$\frac{}{\Gamma, \Gamma_{alias} \vdash x : \Gamma(x)}$$

For function applications (including applications of binary and unary operators), the following two type rules are used, depending on the type of E_0.

$$\frac{\Gamma, \Gamma_{alias} \vdash E_0 : (t_1, \ldots, t_n) \rightarrow t \quad \Gamma, \Gamma_{alias} \vdash E_1 : t'_1, \ldots, \Gamma, \Gamma_{alias} \vdash E_n : t'_n \quad (\forall 1 \leq i \leq n)(t'_i \wedge t_i \neq \emptyset)}{\Gamma, \Gamma_{alias} \vdash E_0 \ \texttt{(}\ E_1, \ldots, E_n\ \texttt{)} : t}$$

$$\frac{\Gamma, \Gamma_{alias} \vdash E_0 : \texttt{any} \quad \Gamma, \Gamma_{alias} \vdash E_1 : t'_1, \ldots, \Gamma, \Gamma_{alias} \vdash E_n : t'_n}{\Gamma, \Gamma_{alias} \vdash E_0 \ \texttt{(}\ E_1, \ldots, E_n\ \texttt{)} : \texttt{any}}$$

The type of the operator must be a function type with the right number of parameters, and the type of every argument must be a success type of the corresponding parameter type of the function type. If all of the conditions are met, the type of the function application is the same as the return type of the function type that is the type of the operator. If the type of the operator is `any`, the return type will be `any`.
Applications of binary and unary operators are treated the same as function applications, with the exception of the `+` operator. We use the $\subseteq$ operator to indicate that a type is a subset of another type, as defined below:

- A type is a subset of type `number` if it is of type `number`, literal number type, or a union type containing any number of literal number types.
- A type is a subset of type `string` if it is of type `string`, literal string type, or a union type containing any number of literal string types.

For the `+` operator, the following rules are applied, in order of priority:

1. If the expression on the left side is a subset of type `number`, check that the other expression is a success type of `number`. The return type is `number`.
2. If the expression on the left side is a subset of type `string`, check that the other expression is a success type of `string`. The return type is `string`.
3. If the expression on the right side is a subset of type `number`, check that the other expression is a success type of `number`. The return type is `number`.
4. If the expression on the right side is a subset of type `string`, check that the other expression is a success type of `string`. The return type is `string`.
5. If the expression on the left side cannot be narrowed to a subset of either `number` or `string`, check that both sides are success types of `number | string`. The return type is `number | string`.

$$\frac{\Gamma, \Gamma_{alias} \vdash E_0 : t_0 \quad \Gamma, \Gamma_{alias} \vdash E_1 : t_1 \quad t_0 \subseteq \texttt{number} \quad t_1 \wedge \texttt{number} \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash E_0 \ + \ E_1 : \texttt{number}}$$

$$\frac{\Gamma, \Gamma_{alias} \vdash E_0 : t_0 \quad \Gamma, \Gamma_{alias} \vdash E_1 : t_1 \quad t_0 \subseteq \texttt{string} \quad t_1 \wedge \texttt{string} \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash E_0 \ + \ E_1 : \texttt{string}}$$

$$\frac{\Gamma, \Gamma_{alias} \vdash E_0 : t_0 \quad \Gamma, \Gamma_{alias} \vdash E_1 : t_1 \quad t_1 \subseteq \texttt{number} \quad t_0 \wedge \texttt{number} \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash E_0 \ + \ E_1 : \texttt{number}}$$

$$\frac{\Gamma, \Gamma_{alias} \vdash E_0 : t_0 \quad \Gamma, \Gamma_{alias} \vdash E_1 : t_1 \quad t_1 \subseteq \texttt{string} \quad t_0 \wedge \texttt{string} \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash E_0 \ + \ E_1 : \texttt{string}}$$

$$\frac{\Gamma, \Gamma_{alias} \vdash E_0 : t_0 \quad \Gamma, \Gamma_{alias} \vdash E_1 : t_1 \quad t_0 \wedge (\texttt{number} \mid \texttt{string}) \neq \emptyset \quad t_1 \wedge (\texttt{number} \mid \texttt{string}) \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash E_0 \ + \ E_1 : \texttt{number} \mid \texttt{string}}$$

For lambda expressions, we temporarily extend Γ with the declared types of all the function parameters, and check the type of the function body against the declared return type. As type syntax is optional, if type annotations are absent for any of the arguments or the return type, the type is assumed to be `any`. The type of the lambda expression is then the function type with the declared types of the parameters and the return type.

$$\frac{\Gamma[x_1 \leftarrow t_1] \cdots [x_n \leftarrow t_n] \vdash S : t' \quad t' \wedge t \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash (x_1 : \ t_1, \ldots, x_n : \ t_n) : \ t \texttt{ => } S : (t_1, \ldots, t_n) \rightarrow t}$$

The type of a conditional expression is the union of the type of its consequent expression and its alternate expression. The predicate expression of a conditional expression must be a success type of a boolean.

$$\frac{\Gamma, \Gamma_{alias} \vdash E_{pred} : t_{pred} \quad \Gamma, \Gamma_{alias} \vdash E_{cons} : t_{cons} \quad \Gamma, \Gamma_{alias} \vdash E_{alt} : t_{alt} \quad t_{pred} \wedge \texttt{boolean} \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash E_{pred} \ \texttt{?} \ E_{cons} : E_{alt} : t_{cons} \mid t_{alt}}$$

For as expressions, the type to cast the expression to must be a success type of the type of the expression.

$$\frac{\Gamma, \Gamma_{alias} \vdash E : t' \quad t \wedge t' \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash E \texttt{ as } t : t}$$

2.3.2 Typing Relations on Statements

Sequences in the top level are handled using the following steps:

1. Type alias declarations are evaluated, which adds type aliases to Γ_{alias} to construct Γ'_{alias}.
2. The declared types of constant declarations are added to Γ to construct Γ'. Note that the declaration statements themselves are yet to be checked.

3. All statements are checked using Γ' and Γ'_{alias}.
4. The type of the sequence is the type of the last value-producing statement.

In the below rule, D_n denotes constant declarations of the form `const` x_n`:` $t_n = E_n$`;`. If the type annotation for x_n is absent, the declared type t_n is assumed to be `any`.

$$\frac{\begin{array}{c}\Gamma_{alias} \vdash A_1 : \Gamma_{alias1}, \ldots, \Gamma_{aliasm-1} \vdash A_m : \Gamma'_{alias} \quad \Gamma[x_1 \leftarrow t_1] \cdots [x_n \leftarrow t_n]\Gamma' \\ \Gamma', \Gamma'_{alias} \vdash D_1 : t_1, \ldots, \Gamma', \Gamma'_{alias} \vdash D_n : t_n \quad \Gamma', \Gamma'_{alias} \vdash S_1 : t'_1, \ldots, \Gamma', \Gamma'_{alias} \vdash S_p : t'_p\end{array}}{\Gamma, \Gamma_{alias} \vdash \{A_1, \ldots, A_m, D_1, \ldots, D_n, S_1, \ldots, S_p\} : t'_p, \Gamma', \Gamma'_{alias}}$$

For type alias declarations, the declared type t for type alias name T is first checked against the type environments. Any type parameters declared are temporarily added to the type alias environment when checking the type of t to ensure that the type parameters are only used within t itself. Then, the binding of T to type function $< T_1, \ldots, T_n > \rightarrow t$ is added to the type alias environment. If no type parameters are given, the type function is assumed to take in 0 type arguments.

$$\frac{\Gamma, \Gamma_{alias} \vdash t : t \quad \Gamma_{alias}[T \leftarrow <> \rightarrow t]\Gamma'_{alias}}{\Gamma, \Gamma_{alias} \vdash \texttt{type}\ T = t\texttt{;} : \texttt{undefined}, \Gamma'_{alias}}$$

$$\frac{\Gamma, \Gamma_{alias}[T_1 \leftarrow T_1] \ldots [T_n \leftarrow T_n] \vdash t : t \quad \Gamma_{alias}[T \leftarrow < T_1, \ldots, T_n > \rightarrow t]\Gamma'_{alias}}{\Gamma, \Gamma_{alias} \vdash \texttt{type}\ T < T_1, \ldots, T_n >= t\texttt{;} : \texttt{undefined}, \Gamma'_{alias}}$$

For constant declarations, the declared type t is retrieved from the type environment. If the declared type is a type reference to a type alias with name T, t is obtained by applying the type arguments $t_1, \ldots, t_n$ to the type function for T, replacing all instances of type variables $T_1, \ldots, T_n$ in t with $t_1, \ldots, t_n$ respectively.
The derived type of the expression E, t_E, must be a success type of t. The type of the statement itself is `undefined`.

$$\frac{\Gamma \vdash E : t_E \quad t_E \wedge t \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash \texttt{const}\ x\texttt{:}\ t = E\texttt{;} : \texttt{undefined}}$$

$$\frac{\Gamma_{alias}(T) < t_1, \ldots, t_n >= t \quad \Gamma \vdash E : t_E \quad t_E \wedge t \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash \texttt{const}\ x\texttt{:}\ T < t_1, \ldots, t_n >= E\texttt{;} : \texttt{undefined}}$$

The type of return statements and expression statements is the type of the expression in the statement.

$$\frac{\Gamma, \Gamma_{alias} \vdash E : t}{\Gamma, \Gamma_{alias} \vdash \texttt{return}\ E\texttt{;} : t} \qquad \frac{\Gamma, \Gamma_{alias} \vdash E : t}{\Gamma, \Gamma_{alias} \vdash E\texttt{;} : t}$$

For blocks, Γ is first extended temporarily to include the types of names declared in the block. Then, the component statements are checked against the extended type environment.
For function body blocks and if statement blocks, we assume that whenever there is a return statement or a conditional statement with a return statement within a block, it is the last statement in the block. (One could consider a “dead code” error otherwise.)

The type of a function body or if statement block is the type of the return statement in the block. If the block does not contain any return statements, the type is `void`, which is a special type that is used to denote the return type of a function that does not return anything, and changes to `undefined` if unioned with another type that is not `void`.
In the below rule, D_n denotes constant declarations of the form `const` x_n`:` $t_n = E_n$`;`. If the type annotation for x_n is absent, the declared type t_n is assumed to be `any`.

$$\frac{\Gamma[x_1 \leftarrow t_1] \cdots [x_m \leftarrow t_m]\Gamma_{temp} \quad \Gamma_{temp} \vdash D_1 : t_1, \ldots, \Gamma_{temp} \vdash D_m : t_m \quad \Gamma_{temp} \vdash S_1 : t'_1, \ldots, \Gamma_{temp} \vdash S_n : t'_n}{\Gamma, \Gamma_{alias} \vdash \{D_1, \ldots, D_m, S_1, \ldots, S_n\} : \begin{cases} t'_n & S_n \textit{ is a return statement} \\ \texttt{void} & S_n \textit{ is not a return statement} \end{cases}}$$

The type of a block that is not a function body or if statement block is the type of last value-producing statement in the block.
In the below rule, D_n denotes constant declarations of the form `const` x_n`:` $t_n = E_n$`;`. If the type annotation for x_n is absent, the declared type t_n is assumed to be `any`. We also assume that S_k is the last value-producing statement in the block.

$$\frac{\Gamma[x_1 \leftarrow t_1] \cdots [x_m \leftarrow t_m]\Gamma_{temp} \quad \Gamma_{temp} \vdash D_1 : t_1, \ldots, \Gamma_{temp} \vdash D_m : t_m \quad \Gamma_{temp} \vdash S_1 : t'_1, \ldots, \Gamma_{temp} \vdash S_n : t'_n}{\Gamma, \Gamma_{alias} \vdash \{D_1, \ldots, D_m, S_1, \ldots, S_n\} : t'_k}$$

The type of a conditional statement or if statement is the union of the type of its consequent statement and its alternate statement. The predicate expression of a conditional statement must be a success type of a boolean.

$$\frac{\Gamma, \Gamma_{alias} \vdash S_{pred} : t_{pred} \quad \Gamma, \Gamma_{alias} \vdash S_{cons} : t_{cons} \quad \Gamma, \Gamma_{alias} \vdash S_{alt} : t_{alt} \quad t_{pred} \wedge \texttt{boolean} \neq \emptyset}{\Gamma, \Gamma_{alias} \vdash \texttt{if}\ (S_{pred})\ S_{cons}\ \texttt{else}\ S_{alt} : t_{cons} \mid t_{alt}}$$

3 Dynamic Type Checking

Expressions evaluate to numbers, boolean values, strings or function values. Implementations of Source generate error messages when unexpected values are used as follows.
Only function values can be applied using the syntax:

expression ::= *name* **(** *expressions* **)**

For compound functions, implementations need to check that the number of *expressions* matches the number of parameters.
The following table specifies what arguments Source's operators take and what results they return. Implementations need to check the types of arguments and generate an error message when the types do not match.

operator	argument 1	argument 2	result
`+`	number	number	number
`+`	string	string	string
`-`	number	number	number
`*`	number	number	number
`/`	number	number	number
`%`	number	number	number
`===`	number	number	bool
`===`	string	string	bool
`!==`	number	number	bool
`!==`	string	string	bool
`>`	number	number	bool
`>`	string	string	bool
`<`	number	number	bool
`<`	string	string	bool
`>=`	number	number	bool
`>=`	string	string	bool
`<=`	number	number	bool
`<=`	string	string	bool
`&&`	bool	any	any
`||`	bool	any	any
`!`	bool		bool
`-`	number		number

Preceding `?` and following `if`, Source only allows boolean expressions.

4 Standard Library

The standard library contains constants and functions that are always available in this language. The functions indicated as *primitive* are built into the language implementations. All others are considered *predeclared* and implemented using the primitive functions.

MISC Library

The following names are provided by the MISC library:

- `get_time()`: *primitive*, returns number of milliseconds elapsed since January 1, 1970 00:00:00 UTC
- `parse_int(s, i)`: *primitive*, interprets the *string* `s` as an integer, using the positive integer `i` as radix, and returns the respective value, see ECMAScript Specification, Section 18.2.5.
- `undefined`, `NaN`, `Infinity`: *primitive*, refer to JavaScript's undefined, NaN ("Not a Number") and Infinity values, respectively.
- `is_boolean(x)`, `is_number(x)`, `is_string(x)`, `is_undefined(x)`, `is_function(x)`: *primitive*, returns `true` if the type of `x` matches the function name and `false` if it does not. Following JavaScript, we specify that `is_number` returns `true` for `NaN` and `Infinity`.
- `prompt(s)`: *primitive*, pops up a window that displays the *string* `s`, provides an input line for the user to enter a text, a "Cancel" button and an "OK" button. The call of `prompt` suspends execution of the program until one of the two buttons is pressed. If the "OK" button is pressed, `prompt` returns the entered text as a string. If the "Cancel" button is pressed, `prompt` returns a non-string value.
- `display(x)`: *primitive*, displays the value `x` in the console[9]; returns the argument `a`.
- `display(x, s)`: *primitive*, displays the string `s`, followed by a space character, followed by the value `x` in the console[9]; returns the argument `x`.

[9]The notation used for the display of values is consistent with JSON, but also displays `undefined` and function objects.

- `error(x)`: *primitive*, displays the value `x` in the console[9] with error flag. The evaluation of any call of `error` aborts the running program immediately.
- `error(x, s)`: *primitive*, displays the string `s`, followed by a space character, followed by the value `x` in the console[9] with error flag. The evaluation of any call of `error` aborts the running program immediately.
- `stringify(x)`: *primitive*, returns a string that represents[9] the value `x`.

All library functions can be assumed to run in $O(1)$ time, except `display`, `error` and `stringify`, which run in $O(n)$ time, where n is the size (number of components such as pairs) of their first argument.

MATH Library

The following names are provided by the MATH library:

- `math_`*name*, where *name* is any name specified in the JavaScript `Math` library, see ECMAScript Specification, Section 20.2. Examples:
  - `math_PI`: *primitive*, refers to the mathematical constant π,
  - `math_sqrt(n)`: *primitive*, returns the square root of the *number* `n`.

All math functions can be assumed to run in $O(1)$ time and are considered *primitive*. All math functions expect numbers as arguments and return numbers. We don't specify the behavior of a math function when some arguments are not numbers.

Deviations from JavaScript

We intend the Source language to be a conservative extension of JavaScript: Every correct Source program should behave *exactly* the same using a Source implementation, as it does using a JavaScript implementation. We assume, of course, that suitable libraries are used by the JavaScript implementation, to account for the predefined names of each Source language. This section lists some exceptions where we think a Source implementation should be allowed to deviate from the JavaScript specification, for the sake of internal consistency and esthetics.

Evaluation result of programs: JavaScript statically distinguishes between *value-producing* and *non-value-producing statements*. All declarations are non-value-producing, and all expression statements, conditional statements and assignments are value-producing. A block is value-producing if its body statement is value-producing, and then its value is the value of its body statement. A sequence is value-producing if any of its component statements is value-producing, and then its value is the value of its *last* value-producing component statement. The value of an expression statement is the value of the expression. The value of a conditional statement is the value of the branch that gets executed, or the value `undefined` if that branch is not value-producing. The value of an assignment is the value of the expression to the right of its `=` sign. Finally, if the whole program is not value-producing, its value is the value `undefined`.

Example 1:

```
1;
{
  // empty block
}
```

The result of evaluating this program in JavaScript is 1.

Example 2:

```
1;
{
  if (true) {} else {}
}
```

The result of evaluating this program in JavaScript is `undefined`.

Implementations of Source are currently allowed to opt for a simpler scheme.

Hoisting of function declarations: In JavaScript, function declarations are "hoisted" (automagically moved) to the beginning of the block in which they appear. This means that applications of functions that are declared with function declaration statements never fail because the name is not yet assigned to their function value. The specification of Source does not include this hoisting; in Source, function declaration can be seen as syntactic sugar for constant declaration and lambda expression. As a consequence, application of functions declared with function declaration may fail in Source if the name that appears as function expression is not yet assigned to the function value it is supposed to refer to.